

Ausführliche Dokumentation des Programms zur Berechnung der Gewinnwahrscheinlichkeiten

1 Spielidee & Regeln

Es gibt m Felder $1, \dots, m$ mit Treffwahrscheinlichkeiten $p_1, \dots, p_m$ pro Wurf, wobei genau ein Feld getroffen wird und $\sum_{j=1}^m p_j = 1$ gilt. Zwei Personen (A und B) legen Chips auf die m Felder:

$$V = (v_1, \dots, v_m), \quad W = (w_1, \dots, w_m).$$

In jedem Wurf wird ein Feld j getroffen. *Falls* dort Chips liegen, darf jede Person *einen* ihrer Chips auf diesem Feld entfernen. Wer zuerst alle eigenen Chips entfernt hat, gewinnt. Räumen beide im selben Wurf alle Chips weg, ist es *Unentschieden*.

1.1 Exakte Berechnung in Python - Umsetzung der Rekursionsformel

Listing 1: Exakte Berechnung - mit Eingabe und Ausgabe

```
1 from fractions import Fraction
2 from functools import lru_cache
3
4 @lru_cache(None)
5 def P_A(V, W):
6     V, W = list(V), list(W)
7     sum_V, sum_W = sum(V), sum(W)
8     if sum_V == 0 and sum_W == 0: return 0 # unentsch.
9     elif sum_V == 0: return 1 # A gewinnt
10    elif sum_W == 0: return 0 # B gewinnt
11
12    s = sum(p for p, v_i, w_i in zip(p_list, V, W) if v_i or w_i)
13
14    prob = 0
15    for i, p in enumerate(p_list):
16        if V[i] or W[i]:
17            V_new, W_new = V.copy(), W.copy()
18            if V_new[i]: V_new[i] -= 1
19            if W_new[i]: W_new[i] -= 1
20            prob += (p/s)*P_A(tuple(V_new), tuple(W_new))
21    return prob
22
23 def compute_outcomes(V, W):
24     PA = P_A(tuple(V), tuple(W))
25     PB = P_A(tuple(W), tuple(V))
26     PU = 1 - PA - PB
27     return PA, PB, PU
28
29 # --- Eingabe von p (p_list), V und W sowie eine Ausgabe ---
30 p_list = (Fraction(1,2), Fraction(1,3), Fraction(1,6))
31 V = (3,2,1)
32 W = (4,2,0)
33
34 PA, PB, PU = compute_outcomes(V, W)
35 print(f"P_A({V},{W}) = {PA} {float(PA):.4f}")
36 print(f"P_B({V},{W}) = {PB} {float(PB):.4f}")
37 print(f"P_U({V},{W}) = {PU} {float(PU):.4f}")
```

2 Beispielausgaben

Für $p_list = (\frac{1}{2}, \frac{1}{3}, \frac{1}{6})$, $\mathbf{V} = (3, 2, 1)$, $\mathbf{W} = (4, 2, 0)$ gilt:

$$\begin{aligned} P(\text{A gewinnt}) &= \frac{1.181.921}{2.400.000} \approx 0,4925, \\ P(\text{B gewinnt}) &= \frac{65}{256} \approx 0,2539, \\ P(\text{Unentschieden}) &= \frac{9.511}{37.500} \approx 0,2536. \end{aligned}$$

Die Leistungsfähigkeit von Python zeigt sich eindrucksvoll, wenn man Gewinnwahrscheinlichkeiten zum Spiel *Differenz trifft* (rekursiv) berechnen lässt.

Für $p_list = (\frac{3}{18}, \frac{5}{18}, \frac{4}{18}, \frac{3}{18}, \frac{2}{18}, \frac{1}{18})$, $\mathbf{V} = (3, 5, 4, 3, 2, 1)$, $\mathbf{W} = (3, 7, 4, 3, 1, 0)$ gilt:

$$\begin{aligned} P(\text{A gewinnt}) &= \frac{3.888.888.135.672.159.681.320.221.494.232.671.796.054.003.132.620.362.357.225.941.957.050.848.818.874.193.779}{12.581.912.148.468.425.737.610.850.489.909.403.379.292.234.521.356.730.060.724.355.620.107.150.426.112.000.000} \approx 0,3091 \\ P(\text{B gewinnt}) &= \frac{717.893.909.052.852.144.591.022.059.467.023.869.946.490.400.857.412.854.873.912.187.561.314.274.728.979.169}{2.180.864.772.401.193.794.519.214.084.917.629.919.077.320.650.368.499.877.192.221.640.818.572.740.526.080.000} \approx 0,3292 \\ P(\text{Unentschieden}) &= \frac{4.226.233.499.099.110.563.389.394.463.126.480.024.007.873.427.804.772.573.566.854.539.473.810.734.958.431.657}{11.683.204.137.863.538.184.924.361.169.201.588.852.199.932.055.545.535.056.386.901.647.242.353.967.104.000.000} \approx 0,3617 \end{aligned}$$

Anhang: Erläuterung der Python-Implementierung zur exakten Berechnung

Codezeile	Beschreibung	Mathematischer Hintergrund	Didaktischer Hinweis
<code>from fractions import Fraction</code>	Exakter Bruchrechnen-Import: erlaubt Eingaben wie <code>Fraction(1,3)</code> und hält alle Rechnungen exakt (keine Rundungsfehler).	Arithmetik in $\mathbb{Q}$ statt in $\mathbb{R}$; Wahrscheinlichkeiten als rationale Zahlen bleiben unter $+$, $-$, $\cdot$, $/$ exakt.	Unterschied <i>float</i> vs. <i>Fraction</i> zeigen (z.B. $1/3$), Begründung für exakte Wahrscheinlichkeitsergebnisse.
<code>from functools import lru_cache</code>	Importiert ein Hilfsmittel aus der Python-Bibliothek.	<code>lru_cache</code> speichert Ergebnisse von Funktionsaufrufen ab.	So lassen sich Zustände wiederverwenden, ohne alles neu zu berechnen.
<code>@lru_cache(None)</code>	Dekorator für die Funktion <code>P_A</code> .	Merkt sich die Ergebnisse für gegebene (V, W) .	Verdeutlicht den Unterschied zwischen mathematischer Idee und effizienter Umsetzung.
<code>def P_A(V, W):</code>	Definiert die Funktion $P_A(V, W)$.	$P_A(V, W)$ ist die Wahrscheinlichkeit, dass A gewinnt, wenn die Restchips V und W verteilt sind.	Übersetzt direkt die mathematische Notation in ein Python-Programm.
<code>V, W = list(V), list(W)</code>	Wandelt Tupel in Listen um.	Listen können verändert werden (Chips abziehen).	Schüler:innen lernen den Unterschied zwischen fester und veränderbarer Datenstruktur.
<code>sum_V, sum_W = sum(V), sum(W)</code>	Summiert die Chipzahlen von A und B.	Entspricht $\sum V_i$ bzw. $\sum W_i$.	Zeigt: Wir müssen nur die Gesamtzahl betrachten, um Basisfälle zu prüfen.
<code>if sum_V == 0 and sum_W == 0: return 0</code>	Prüft: Beide haben keine Chips mehr.	Das Spiel endet unentschieden, $P_A = 0$.	Einfache Abbruchbedingung, auch im Unterricht gut nachvollziehbar.
<code>elif sum_V == 0: return 1</code>	Prüft: A ist fertig, B nicht.	A gewinnt sicher.	Veranschaulicht den ersten Gewinnfall.
<code>elif sum_W == 0: return 0</code>	Prüft: B ist fertig, A nicht.	B gewinnt sicher.	Zweiter Gewinnfall. Schüler:innen sehen: Die Basisfälle decken alle Möglichkeiten ab.
<code>s = sum(p for p, v_i, w_i in zip(p_list, V, W) if v_i or w_i)</code>	Normierung über relevante Felder.	$s = \sum_j p_j \cdot 1(v_j > 0 \text{ oder } w_j > 0)$.	Wichtig: Leertreffer werden ausgeschlossen, alles steckt in dieser einen Zeile.

Codezeile	Beschreibung	Mathematischer Hintergrund	Didaktischer Hinweis
<code>prob = 0</code>	Startwert für die Wahrscheinlichkeit.	Hier wird später über die Trefferfelder aufaddiert.	Schüler:innen sehen: Wir berechnen eine Summe schrittweise.
<code>for i, p in enumerate(p_list):</code>	Schleife über alle Felder.	Jedes Feld i wird mit Wahrscheinlichkeit p_i getroffen.	Verdeutlicht den Bezug zur Wahrscheinlichkeitsverteilung p .
<code>if V[i] or W[i]:</code>	Prüft: Gibt es Chips auf Feld i ?	Nur relevante Felder tragen etwas zur Summe bei.	Entspricht der Bedingung $v_i > 0 \vee w_i > 0$.
<code>V_new, W_new = V.copy(), W.copy()</code>	Kopiert die Chip-Verteilungen.	Neue Zustände V', W' nach dem Treffer.	Vermeidet, dass der ursprüngliche Zustand verändert wird.
<code>if V_new[i]: V_new[i] -= 1</code>	Zieht ggf. einen Chip von A ab.	$V'_i = V_i - 1$, falls $V_i > 0$.	Klarer Bezug zur Spielregel: A nimmt einen Chip weg.
<code>if W_new[i]: W_new[i] -= 1</code>	Zieht ggf. einen Chip von B ab.	$W'_i = W_i - 1$, falls $W_i > 0$.	Analog für B: B nimmt einen Chip weg.
<code>prob += (p/s) * P_A(tuple(V_new), tuple(W_new))</code>	Rekursiver Aufruf mit neuer Verteilung.	$P_A(V, W) = \sum \frac{p_i}{s} \cdot P_A(V', W')$.	Der zentrale Schritt: Formel wird 1:1 umgesetzt.
<code>return prob</code>	Gibt die berechnete Wahrscheinlichkeit zurück.	$P_A(V, W)$ ist nun vollständig bestimmt.	Rückkehr zur oberen Ebene der Rekursion.
<code>def compute_outcomes(V, W):</code>	Zweite Funktion: berechnet alle drei Wahrscheinlichkeiten.	Nutzt $P_A(V, W)$ und $P_A(W, V)$.	Betont die Symmetrie zwischen den Spielern.
<code>PA = P_A(tuple(V), tuple(W))</code>	Wahrscheinlichkeit, dass A gewinnt.	$P_A(V, W)$.	Direktes Ergebnis aus der Hauptfunktion.
<code>PB = P_A(tuple(W), tuple(V))</code>	Wahrscheinlichkeit, dass B gewinnt.	$P_B(V, W) = P_A(W, V)$.	Einfach durch Rollentausch berechnet.
<code>PU = 1 - PA - PB</code>	Wahrscheinlichkeit für Unentschieden.	$P_U = 1 - P_A - P_B$.	Erinnert an Vollständigkeit: Alle Fälle sind abgedeckt.
<code>return PA, PB, PU</code>	Rückgabe aller drei Wahrscheinlichkeiten.	(P_A, P_B, P_U) .	Das Endergebnis: genau die Werte, nach denen man fragt.

Codezeile	Beschreibung	Mathematischer Hintergrund	Didaktischer Hinweis
<code>p_list = (Fraction(1,2), Fraction(1,3), Fraction(1,6))</code>	Definition der Treffwahrscheinlichkeiten als Tupel rationaler Zahlen.	$p_j \geq 0$ und $\sum_j p_j = 1$; hier $1/2 + 1/3 + 1/6 = 1$.	Normierung prüfbar; Fraction macht die exakte Summe sichtbar.
<code>V = (3, 2, 1)</code>	Startverteilung der Chips von A über die m Felder.	$V \in \mathbb{N}_0^m$; $\sum_i V_i =$ Gesamtzahl Chips von A.	Konkrete Parametrisierung der Spielsituation erleichtert die Interpretation.
<code>W = (4, 2, 0)</code>	Startverteilung der Chips von B über die m Felder.	$W \in \mathbb{N}_0^m$; $\sum_i W_i =$ Gesamtzahl Chips von B.	Ergänzt das vollständige Anfangsszenario für die Berechnung.
<code>PA, PB, PU = compute_outcomes(V, W)</code>	Aufruf der Auswertungsfunktion; liefert Gewinnwahrscheinlichkeiten für A, B und Unentschieden.	Verwendet $P_U = 1 - P_A - P_B$ und Symmetrie $P_B(V, W) = P_A(W, V)$.	Saubere Trennung: Berechnung (Funktion) vs. Daten (oben) vs. Darstellung (unten).
<code>print(f"P_A({V},{W}) = {PA} ≈ {float(PA):.4f}")</code>	Gibt P_A sowohl exakt (Bruch) als auch als Dezimalapproximation aus.	Zwei Repräsentationen desselben Werts; Rundung nur in der Anzeige.	Fördert Verständnis: Exaktheit vs. numerische Darstellung.
<code>print(f"P_B({V},{W}) = {PB} ≈ {float(PB):.4f}")</code>	Analog für P_B .		Vergleich der Gewinnchancen beider Spieler wird direkt sichtbar.
<code>print(f"P_U({V},{W}) = {PU} ≈ {float(PU):.4f}")</code>	Analog für P_U ; komplettiert die Ausgabe.	Prüfbar: $P_A + P_B + P_U = 1$ (bis auf Rundung in der Anzeige).	Einladung zur Plausibilitätskontrolle (Summentest ≈ 1).